Campus L&F: Aplicação Android para Registo e Consulta de Objetos Perdidos

Afonso Maia (1242115) · Fernanda Vale (1251223)

Licenciatura em Engenharia de Telecomunicações e Informática
Instituto Superior de Engenharia do Porto · DSSMV · Turma 2DB · 2026/2027

Resumo. O Campus L&F é uma aplicação Android nativa, desenvolvida em Java, destinada ao registo e à consulta de objetos encontrados no Campus do ISEP. A solução permite fotografar um objeto, selecionar a sua categoria e o edifício, indicar a data e o local do achado, associar opcionalmente coordenadas GPS e publicar o registo num catálogo partilhado através da API RESTDB. A aplicação disponibiliza pesquisa e filtros, apresenta os detalhes de cada objeto e mantém uma cache local Room/SQLite para garantir continuidade de utilização perante falhas temporárias de rede. A arquitetura segue o padrão MVVM, separando View, ViewModel, Repository, fontes de dados remota e local, modelo de domínio e serviços do dispositivo.

Palavras-chave: Android, Java, MVVM, RESTDB, Retrofit, Room, GPS, Câmara, UML, objetos perdidos

1 Contexto

No Campus do ISEP é frequente existirem objetos encontrados sem uma forma centralizada de divulgação. O Campus L&F resolve este problema através de um catálogo partilhado, permitindo que qualquer utilizador publique um achado e que os restantes utilizadores consultem os registos para reconhecer um objeto perdido.
O projeto cumpre o objetivo do ProjectDroid ao desenvolver uma aplicação Android em Java que comunica com serviços externos e explora funcionalidades do dispositivo móvel. A versão final integra RESTDB, fotografia, localização GPS opcional, pesquisa, filtros e persistência local. O fluxo principal consiste em encontrar um objeto, fotografá-lo, preencher os dados, publicá-lo e disponibilizá-lo imediatamente aos restantes utilizadores.

1.1 Objetivos e Funcionalidades

- Publicar objetos encontrados com fotografia, título, descrição, categoria, edifício, localização detalhada e data do achado.
- Associar opcionalmente a posição GPS atual ao registo, após autorização explícita do utilizador.

- Consultar o catálogo partilhado e visualizar todos os detalhes de um objeto.
- Pesquisar e filtrar objetos por texto, categoria e edifício.
- Sincronizar os dados com RESTDB e manter uma cache Room/SQLite para consulta dos últimos dados disponíveis quando a rede não está acessível.
- Tratar de forma controlada erros de validação, rede, API, upload de media, persistência, permissões e localização.

2 Análise

Nessa secção será abordado o contexto, as histórias de utilizador, os requisitos e modelo de domínio. As histórias de utilizador descrevem o valor pretendido pelo utilizador, enquanto o modelo de domínio representa os conceitos do problema independentemente das classes técnicas utilizadas na implementação.

2.1 Histórias de Utilizador

Existe um único ator funcional, o Utilizador do Campus. A aplicação não exige autenticação: a mesma pessoa pode consultar o catálogo e publicar um objeto encontrado.

ID	História de Utilizador
US01	Como utilizador do campus, quero consultar os objetos encontrados mais recentes, para poder identificar um objeto que perdi.
US02	Como utilizador do campus, quero ver a fotografia e todos os detalhes de um objeto encontrado, para confirmar se o objeto me pertence.
US03	Como utilizador que encontrou um objeto, quero registá-lo na aplicação, para que outras pessoas o possam encontrar.
US04	Como utilizador que encontrou um objeto, quero tirar ou selecionar uma fotografia com o telemóvel, para tornar o objeto mais fácil de reconhecer.
US05	Como utilizador que encontrou um objeto, quero selecionar a categoria e o edifício, para que a publicação fique corretamente classificada.
US06	Como utilizador que encontrou um objeto, quero associar opcionalmente a localização GPS atual, para descrever com maior precisão o local do achado.
US07	Como utilizador do campus, quero pesquisar e filtrar os objetos por texto, categoria e edifício, para reduzir rapidamente a lista de possíveis correspondências.
US08	Como utilizador do campus, quero que uma publicação seja partilhada através do catálogo remoto, para ficar disponível noutros dispositivos.
US09	Como utilizador do campus, quero consultar os últimos objetos sincronizados quando não existe rede, para que a aplicação continue útil em modo offline.
US10	Como utilizador do campus, quero receber mensagens claras de validação, erro e nova tentativa, para recuperar de uma falha sem perder os dados introduzidos.

2.2 Requisitos Funcionais

ID	Requisito
RF01	A aplicação deve obter da API e apresentar a lista de objetos encontrados.
RF02	A aplicação deve apresentar fotografia, título, descrição, categoria, edifício, data e localização de cada objeto.
RF03	O utilizador deve conseguir criar e publicar um novo objeto encontrado.
RF04	O formulário deve permitir tirar ou selecionar uma fotografia e enviá-la para o serviço de media.
RF05	O formulário deve obter as categorias e os edifícios existentes na API.
RF06	A aplicação deve permitir recolher opcionalmente as coordenadas atuais do dispositivo, após autorização.
RF07	A aplicação deve permitir pesquisar e filtrar objetos por texto, categoria e edifício.
RF08	Os dados remotos devem ser convertidos para objetos de domínio e guardados numa cache Room/SQLite.
RF09	Na ausência de rede, a aplicação deve apresentar a última informação sincronizada disponível localmente.
RF10	A aplicação deve validar os dados e apresentar mensagens específicas para erros de validação, rede, API, media, persistência, permissões e localização.

2.3 Requisitos Não Funcionais

- Usabilidade: o fluxo de publicação deve ser curto, coerente e acompanhado por feedback durante carregamento, upload, publicação e erro.
- Desempenho: operações de rede, imagem e persistência não podem bloquear a thread principal da aplicação.
- Fiabilidade: falhas remotas não devem provocar o encerramento da aplicação; os dados sincronizados devem permanecer disponíveis na cache local.
- Privacidade: a localização GPS é opcional e apenas é recolhida mediante permissão explícita do utilizador.
- Segurança: toda a comunicação remota deve utilizar HTTPS e a chave RESTDB deve possuir apenas as permissões necessárias.
- Portabilidade: o projeto utiliza Gradle Wrapper e Java 17, permitindo compilação consistente em Windows e macOS.
- Manutenibilidade: a solução segue MVVM e separa View, ViewModel, Repository, fontes de dados, modelo e serviços do dispositivo.

2.4 Modelo de Domínio

O conceito central do domínio é ObjetoEncontrado. Cada objeto pertence a uma Categoria e a um Edifício, possui uma ou mais Fotografias e pode incluir uma

LocalizaçãoGeográfica. A localização textual continua obrigatória, enquanto as coordenadas são opcionais.

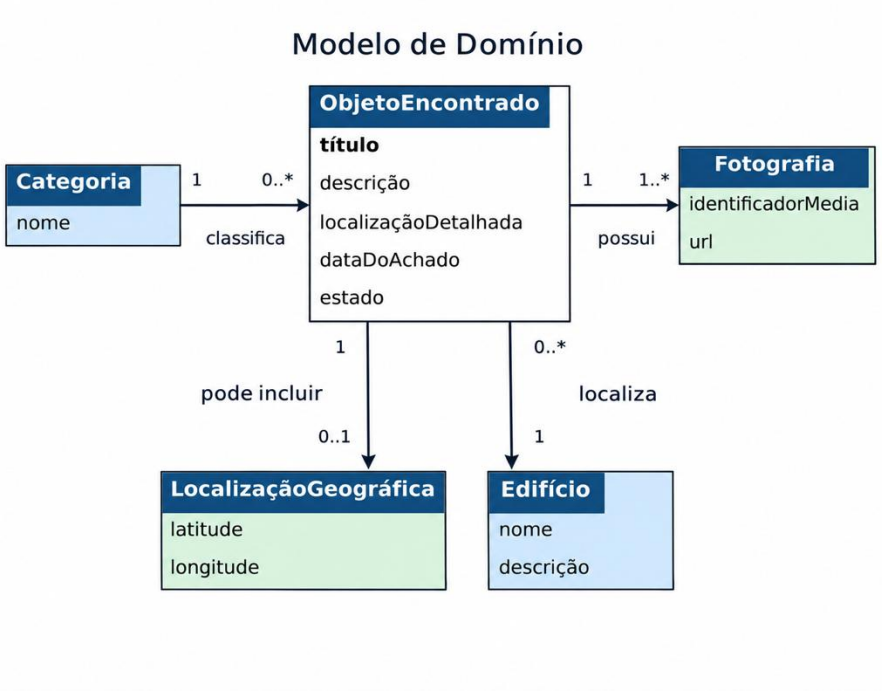


Figura 1. Modelo de domínio do Campus L&F.

2.5 Especificação do UC03 - Registar Objeto Encontrado

Campo	Especificação
Descrição	Publicar um objeto encontrado no catálogo partilhado.
Pré-condições	A aplicação encontra-se aberta. Para publicação imediata deve existir conectividade. A fotografia e a localização GPS dependem das permissões respetivas quando utilizadas.
Pós-condições	É criado um novo objeto no RESTDB e a cache local fica sincronizada com o registo publicado.
Caminho básico	1. O utilizador abre o ecrã Registar objeto. 2. Tira ou seleciona uma fotografia. 3. Introduz título, descrição, categoria, edifício, data e localização detalhada. 4. Opcionalmente solicita a localização GPS. 5. Seleciona Publicar objeto. 6. O sistema valida os dados. 7. O sistema envia a fotografia para o serviço de media. 8. O sistema publica o objeto na API.

Campo	Especificação
	9. O sistema atualiza a cache Room/SQLite. 10. O sistema apresenta sucesso e regressa à lista atualizada.
Caminhos alternativos	Dados inválidos: o sistema identifica os campos incorretos. Permissão recusada: a câmara ou o GPS não são utilizados até o utilizador autorizar. Falha de rede ou API: o formulário é preservado e é apresentada a opção de nova tentativa. Falha de upload ou persistência: a operação é cancelada de forma controlada e é apresentada uma mensagem adequada.

3 Modelos

Nessa secção serão apresentados os modelos de arquitetura física através de um diagrama de deployment, arquitetura lógica através de componentes, diagramas de classes, diagrama de sequência e representação da interface. A solução final adota MVVM, com View, ViewModel, Repository, fontes de dados remota e local, modelo de domínio e serviços do dispositivo.

3.1 Arquitetura Física

A aplicação Campus L&F é instalada num ou mais dispositivos Android. Em cada dispositivo são utilizados a câmara, o serviço de localização e uma base de dados Room/SQLite. A aplicação comunica por HTTPS com o RESTDB, onde se encontram a API, as coleções de dados e o armazenamento das fotografias.

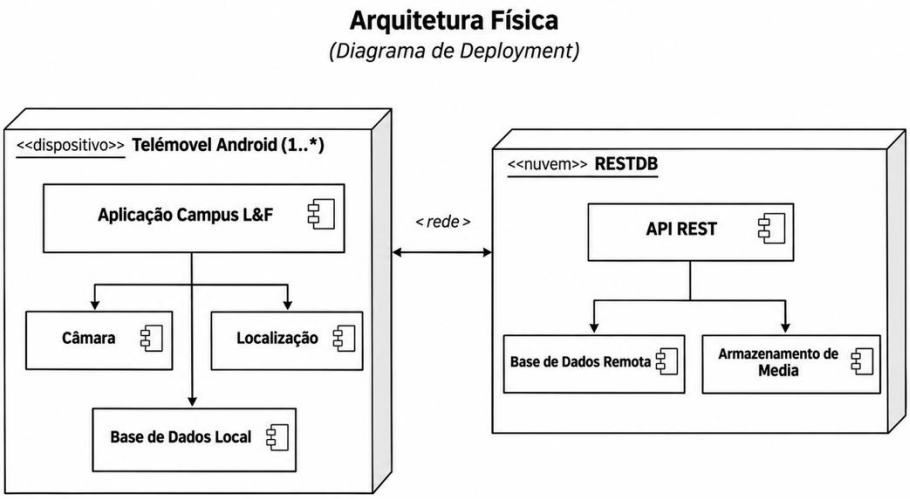


Figura 2. Arquitetura física da aplicação.

3.2 Arquitetura Lógica

A arquitetura lógica segue o padrão MVVM. A View contém Activities, layouts XML e Adapter; os ViewModels mantêm o estado da interface e coordenam os casos de uso; o Repository constitui a fonte única de dados; as fontes remota e local comunicam, respetivamente, com RESTDB e Room/SQLite. Os serviços do dispositivo encapsulam câmara, localização e permissões.

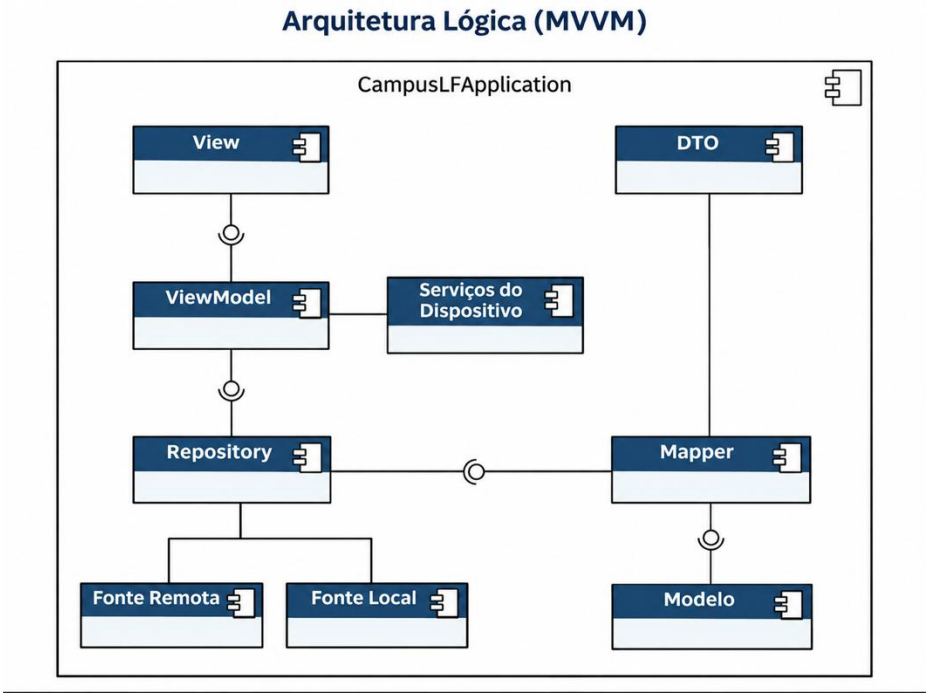


Figura 3. Arquitetura lógica MVVM do Campus L&F.

3.3 Diagrama de Classes do Modelo

O diagrama distingue o modelo de domínio dos objetos de transferência e persistência. FoundItem agrega Category, Building, Photo e uma GeoLocation opcional. RemoteFoundItemDto representa os dados recebidos da API, NewFoundItemDto representa os dados enviados numa publicação, FoundItemEntity representa a cache local e FoundItemMapper realiza as conversões entre estas representações.

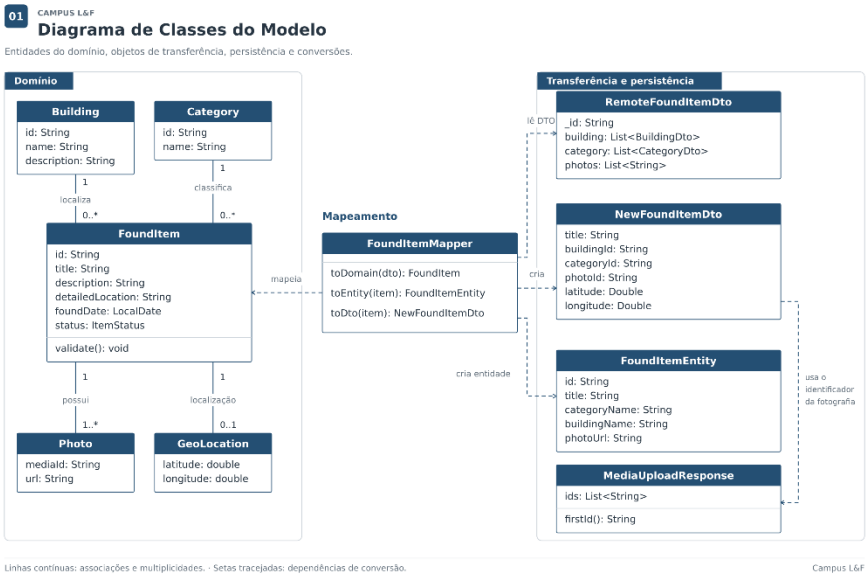


Figura 4. Diagrama de classes do modelo, DTOs e entidade local.

3.4 Diagrama de Classes das Exceções

O tratamento de erros é uniformizado pela classe base CampusLfException. As subclasses distinguem dados inválidos, indisponibilidade de rede, respostas HTTP inválidas, falhas no upload de media, problemas de persistência, falhas de localização e permissões recusadas. A ViewModel converte estas exceções em mensagens adequadas para a interface.

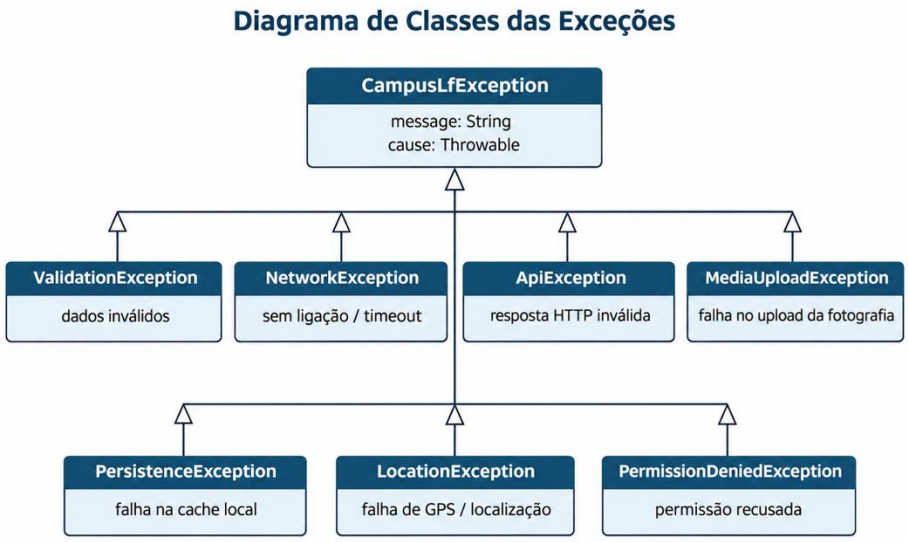


Figura 5. Hierarquia de exceções da aplicação.

3.5 Diagrama Global de Classes

A visão global apresenta as principais classes organizadas por camada. As Activities dependem dos respetivos ViewModels; os ViewModels usam FoundItemRepository; o Repository coordena RestDbDataSource, LocalDataSource e FoundItemMapper. AddItemViewModel utiliza ainda CameraService e LocationService. A fonte remota usa RestDbApi, enquanto a fonte local usa FoundItemDao e CampusDatabase.

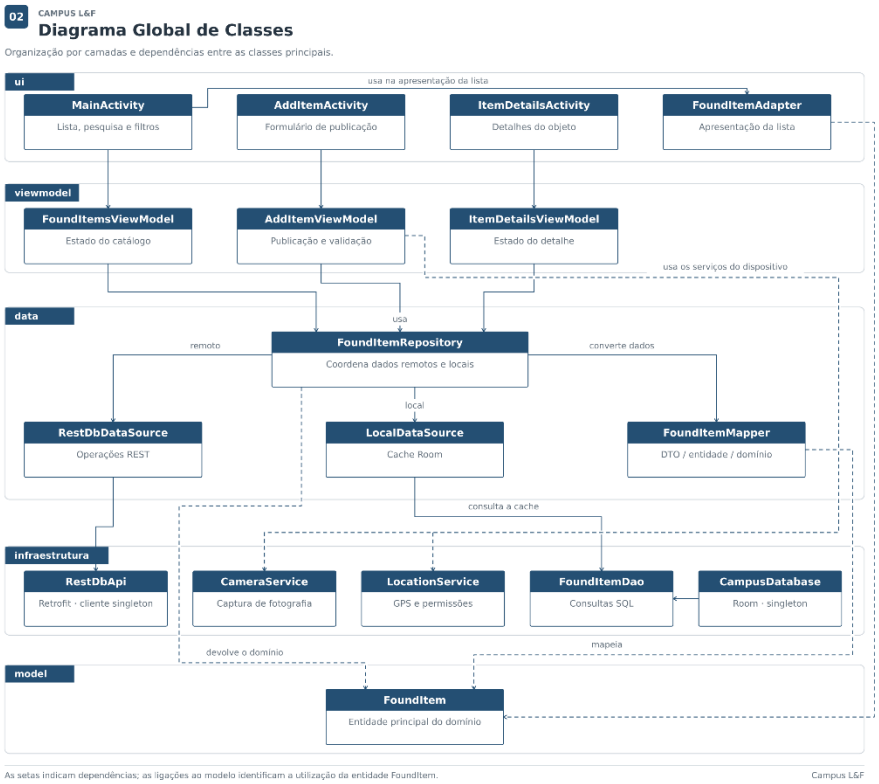


Figura 6. Diagrama global de classes da solução final.

3.6 Diagrama de Sequência - UC03 Registar Objeto Encontrado

O diagrama de sequência detalha o caso de uso de publicação. Depois da validação, a localização é obtida apenas quando solicitada. O Repository envia a fotografia, publica os dados no RESTDB e atualiza a cache local. Os caminhos alternativos preservam o formulário quando existe erro de validação, rede, API, media ou persistência.

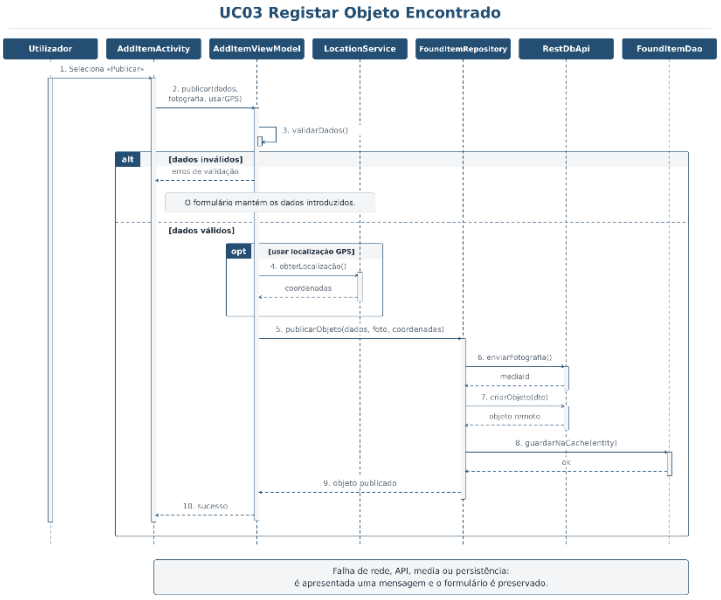


Figura 7. Diagrama de sequência do UC03 – Registar Objeto Encontrado.

3.7 Interface do Utilizador

A interface final utiliza três ecrãs principais. O Catálogo apresenta pesquisa, filtros e a lista dos objetos; o ecrã Registar objeto permite adicionar fotografia, dados, categoria, edifício e GPS; o ecrã Detalhes do objeto apresenta toda a informação e permite consultar a localização. A navegação é direta e mantém o fluxo principal visível ao utilizador.

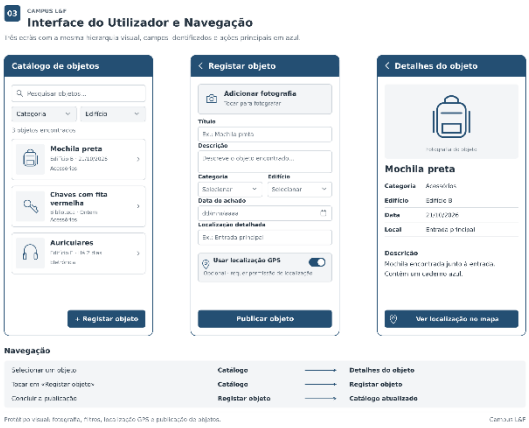


Figura 8. Ecrãs principais e fluxo de navegação.

4 Implementação

A aplicação final é implementada em Java sob o namespace pt.isep.dssmv.campuslf. O projeto utiliza AndroidX, ViewModel e LiveData para MVVM, Retrofit 2 e Gson para comunicação REST, Glide para carregamento de imagens, Room para persistência local e Gradle Wrapper 8.13 para construção consistente do projeto.
- View: MainActivity, AddItemActivity e ItemDetailsActivity apresentam os dados e encaminham ações para os ViewModels.
- ViewModel: FoundItemsViewModel, AddItemViewModel e ItemDetailsViewModel mantêm o estado da interface e coordenam os casos de uso.
- Repository: FoundItemRepository centraliza o acesso aos dados e define a política de sincronização entre RESTDB e a cache local.
- Fonte remota: RestDbApi é criada por um cliente Retrofit singleton e disponibiliza listagem, detalhe, criação, categorias, edifícios e upload de media.
- Fonte local: CampusDatabase é uma base de dados Room singleton; FoundItemDao executa as operações SQL sobre a cache.
- Mapeamento: FoundItemMapper converte DTOs remotos e entidades locais em objetos do domínio.
- Câmara e media: CameraService usa a API de Activity Result para tirar ou selecionar uma fotografia e o Repository envia-a para RESTDB Media.
- Localização: LocationService solicita permissões em runtime e obtém latitude e longitude quando o utilizador ativa a opção GPS.
- Concorrência: RxJava/Executors executam operações de rede, imagem e persistência fora da thread principal; LiveData publica o resultado na interface.
- Erros: a hierarquia CampusLfException permite apresentar feedback coerente sem expor detalhes técnicos.

5 Testes

A validação da versão final combina testes unitários, testes de integração, testes de interface e testes em dispositivo real. Os testes do domínio verificam limites de texto, datas futuras e construção válida de FoundItem. Os testes do Repository validam mapeamento REST, respostas HTTP, sincronização Room e fallback offline. Os testes de interface cobrem navegação, validação do formulário, pesquisa, filtros, fotografia, publicação e permissões de localização.

Tipo	Exemplos de cenários
Unitários	FoundItem válido; campos inválidos; data futura; mapper DTO ↔ domínio; ViewModels; exceções específicas.
Integração	GET/POST no RESTDB; upload de fotografia; categorias e edifícios; sincronização Room; fallback offline.
Interface	Abrir detalhe; pesquisar e filtrar; validação visual; publicar; nova tentativa após falha; permissões de câmara e localização.

Tipo	Exemplos de cenários
Dispositivo real	Câmara, GPS, Wi-Fi/dados móveis e ciclo completo publicar → sincronizar → consultar num segundo dispositivo.

6 Conclusão

O Campus L&F oferece um fluxo completo para publicação e consulta de objetos encontrados no campus. A combinação de catálogo remoto, fotografia, classificação por categoria e edifício, localização opcional e cache local transforma a aplicação numa solução móvel partilhada e não limitada ao próprio dispositivo. A arquitetura MVVM, o Repository e o tratamento uniforme de exceções permitem evoluir a solução sem acoplar a interface aos detalhes da API, persistência ou hardware.

Referências

[1] DSSMV. ProjectDroid – Mobile Android Project, LETI/ISEP, 2026/2027.
[2] Sousa, P. B.; Freitas, C. Requirements Engineering/Analysis, FSOFT TP, Week 4, LETI/ISEP, 2025/26.
[3] Sousa, P. B. Design, FSOFT TP, Week 7, LETI/ISEP, 2025/26.
[4] Android Developers. Documentação da plataforma Android.
[5] Square. Retrofit – cliente HTTP tipado para Android e Java.
[6] RESTDB. Documentação da API REST e do serviço de media.